

Nesneye Yönelik Programlama

Hafta 11

synchronized anahtar kelimesi

Java'daki `synchronized` anahtar kelimesi, en temel haliyle çoklu iş parçacıklarının (multi-threading) aynı anda tek bir kaynağa erişmesini engelleyen bir "kilit" (lock/mutex) mekanizmasıdır.

Birden fazla thread aynı anda paylaşılan bir veriyi (örneğin bir değişkeni, nesneyi veya veritabanı kaydını) okuyup değiştirmeye çalıştığında "Race Condition" (Yarış Durumu) dediğimiz veri tutarsızlıkları ortaya çıkar.

`synchronized` kullanıldığında, bir thread o metoda veya kod bloğuna girdiğinde kapıyı arkasından kilitler. İşini bitirip çıkana kadar diğer tüm thread'ler kapıda beklemek zorundadır.

Thread

Thread (İş Parçacığı), bir işletim sisteminde işlemci (CPU) tarafından bağımsız olarak yönetilebilen ve çalıştırılabilen en küçük komut dizisidir.

Konuyu daha iyi çerçevelemek için "Process" (Süreç) kavramı ile birlikte düşünmek gerekir. Bilgisayarda bir programı (örneğin bir Java uygulamasını) çalıştırdığında, işletim sistemi bu program için hafızada izole bir alan ayırır ve buna Process denir. Her process, çalışmaya başladığında en az bir tane ana thread'e (Main Thread) sahiptir.

Java'daki `public static void main` metodunun çalışması tam olarak bu ana thread'in görevidir.

Thread

Bir programı (Process) koca bir **restoran mutfağı** olarak düşünebilirsiniz. Mutfaktaki her bir aşçı ise birer **Thread**'dir.

- Eğer mutfakta tek bir aşçı (Single-thread) varsa, o aşçı önce çorbayı yapar, o bitince yemeğe geçer, o bitince tatlıyı hazırlar. İşler sırayla (senkron) ilerler.
- Eğer mutfağa yeni aşçılar alırsan (Multi-threading - `ali.start()`, `ayse.start()` vb.), biri çorbayı karıştırırken diğeri aynı anda tatlıyı yapabilir (asenkron). Böylece siparişler çok daha hızlı çıkar.

Thread

Thread'lerin Temel Özellikleri

- **Ortak Hafıza (Shared Memory):** Aynı mutfaktaki aşçıların aynı tezgahı, aynı buzdolabını ve aynı bıçakları kullanması gibi, bir process içindeki tüm thread'ler aynı hafıza alanını (değişkenleri, nesneleri) ortaklaşa kullanır. İşte tam da bu ortak kullanım nedeniyle, iki aşçının aynı anda aynı bıçağı almaya çalışıp birbirini engellememesi için **synchronized** gibi kilit mekanizmalarına ihtiyaç duyarız.
- **Hafiflik (Lightweight):** İşletim sistemi için yeni bir mutfak (Process) inşa etmek çok maliyetli ve yavaştır. Ancak mevcut mutfağın içine yeni bir aşçı (Thread) işe almak çok daha az sistem kaynağı tüketir.
- **Eşzamanlılık (Concurrency) ve Geçiş (Context Switching):** Eğer bilgisayarın işlemcisi tek çekirdekliyse, aşçılar aslında tam olarak "aynı milisaniyede" çalışmaz. İşletim sistemi aşçılar (thread'ler) arasında o kadar hızlı geçiş yapar ki, biz dışarıdan bakarken hepsinin aynı anda çalıştığı illüzyonuna kapılırız. Çok çekirdekli işlemcilerde ise thread'ler gerçekten fiziksel olarak aynı anda (Parallelism) çalışabilir.

synchronized anahtar kelimesi

Yazıcı Simülasyonu: Ofisteki tek yazıcıya 3 kişi aynı anda belge göndermeye çalıştığında, yazıcı korunmazsa yazılar birbirine karışır. **synchronized** kelimesi yazıcıya "sırayla gir" kuralı koyar. Eğer bu kelime kaldırılırsa, çıktılar anlamsız bir şekilde karışacaktır.

Bilet Satış Simülasyonu: 10 bilet satılan bir konserde, **synchronized** kullanılmazsa iki farklı gişe aynı anda son bileti görüp satabilir ve 11 bilet satılmış olur; hatta bilet sayısı -1'e düşebilir. Bilet havuzundaki kontrol metoduna **synchronized** eklemek, her seferinde sadece bir gişenin bilet olup olmadığını kontrol edip satış yapmasını sağlar. Özetle, bu yapı sayesinde her bilet sadece bir kez satılır ve toplam sayı doğru kalır.

Örnek 1: Yazıcı Simülasyonu (Tek Kaynak Paylaşımı)

"Diyelim ki ofiste tek bir yazıcı var ve 3 kişi aynı anda belge göndermeye çalışıyor. Yazıcı aynı anda sadece bir belgeyi basabilir. Eğer yazıcıyı korumazsak, 3 kişinin yazıları birbirine karışır; Ali'nin cümlesinin yarısı, sonra Ayşe'nin cümlesi, sonra Ali'nin kalanı... gibi. İşte synchronized ile yazıcıya 'sırayla gir' kuralı koyacağız."

Örnek 1: Yazıcı Simülasyonu (Tek Kaynak Paylaşımı)

```
1 // Ortak kaynak: Yazıcı
2 class Yazici {
3
4     public synchronized void yazdir(String kullanıcıAdi, String mesaj) {
5         System.out.print(kullanıcıAdi + " yazıyor: ");
6
7         for (char c : mesaj.toCharArray()) {
8             System.out.print(c);
9
10            try {
11                // Thread.sleep(10) ile yazıcının harfleri kağıda basma süresini simüle ediyoruz.
12                // Eğer metot synchronized olmasaydı, bir thread uykuya daldığı bu 10 milisaniyelik
13                // boşlukta işletim sistemi diğer thread'e geçiş (Context Switching) yapacak ve
14                // harfler birbirine girecekti.
15                Thread.sleep(10);
16            } catch (InterruptedException e) {
17                // Thread uykudayken dışarıdan zorla kesintiye uğratılırsa bu hata fırlatılır.
18                System.out.println("Yazdırma sırasında hata oluştu.");
19            }
20        }
21        System.out.println(" [BİTTİ]");
22    }
23 }
```


Örnek 1: Yazıcı Simülasyonu (Tek Kaynak Paylaşımı)

```
25 // Görev sınıfı: Kullanıcı
26 // Java'da bir sınıfın eşzamanlı çalışabilen bir iş parçacığı olabilmesi için
27 // 'Thread' sınıfından kalıtım alması (extends) gerekir.
28 class Kullanici extends Thread {
29     private String ad;
30     private Yazici yazici;
31     private String belge;
32
33     public Kullanici(String ad, Yazici yazici, String belge) {
34         this.ad = ad;
35         this.yazici = yazici;
36         this.belge = belge;
37     }
38
39     // run() metodu, bir thread'in yaşamsal döngüsünün başladığı kalbidir.
40     // Thread başlatıldığında arka planda çalışacak olan kodlar buraya yazılır.
41     @Override
42     public void run() {
43         yazici.yazdir(ad, belge);
44     }
45 }
46
```

Örnek 1: Yazıcı Simülasyonu (Tek Kaynak Paylaşımı)

```
47 // Ana sınıf
48 public class YaziciSimulasyonu {
49     public static void main(String[] args) {
50
51         // KRİTİK NOKTA: Hafızada sadece tek bir Yazici nesnesi oluşturuluyor.
52         Yazici yazici = new Yazici();
53
54         // Aynı yazici referansı, 3 farklı thread'e parametre olarak gönderiliyor.
55         // Synchronized kilidinin çalışabilmesi için herkesin aynı nesneyi kullanması şarttır.
56         Kullanici ali = new Kullanici("Ali", yazici, "Merhaba, bugün hava çok güzel.");
57         Kullanici ayse = new Kullanici("Ayşe", yazici, "Java öğreniyorum ve çok eğleniyorum.");
58         Kullanici mehmet = new Kullanici("Mehmet", yazici, "Threadler konusunu anladım!");
59
60         // start() metodu, run() metodunu çağırmaktan farklıdır.
61         // run() dersek normal bir metot gibi sırayla (senkron) çalışırlar.
62         // start() dediğimizde ise Java Sanal Makinesi (JVM) her biri için yeni bir
63         // çağrı yığını (call stack) oluşturur ve hepsini eşzamanlı (asenkron) olarak başlatır.
64         ali.start();
65         ayse.start();
66         mehmet.start();
67     }
68 }
```

Örnek 1: Yazıcı Simülasyonu (Tek Kaynak Paylaşımı)

Çıktı (synchronized VAR):

Ali yazıyor: Merhaba, bugün hava çok güzel. [BİTTİ]

Ayşe yazıyor: Java öğreniyorum ve çok eğleniyorum. [BİTTİ]

Mehmet yazıyor: Threadler konusunu anladım! [BİTTİ]

Sıra sizde; Konser bileti

Bir konsere 10 bilet satılıyor. 3 gişe (thread) aynı anda bilet satmaya çalışıyor. Eğer synchronized kullanmazsak, iki gişe aynı anda son bileti görebilir ve ikisi de satar → 11 bilet satılmış olur!

Hatta bilet -1'e düşebilir.

synchronized ile her seferinde sadece bir gişe 'bilet var mı?' diye kontrol eder ve satış yapar. Bu programı basitçe simüle ediniz.

```
1 // Ortak kaynak: Bilet havuzu
2 class BiletHavuzu {
3     private int kalanBilet = 10;
4
5     // synchronized ile aynı anda sadece bir thread bilet alabilir
6     public synchronized boolean biletAl(String gişeAdi) {
7         if (kalanBilet > 0) {
8             System.out.println(gişeAdi + " bilet aldı. Kalan bilet: " + (kalanBilet - 1));
9             kalanBilet--;
10            return true;
11        } else {
12            System.out.println(gişeAdi + " bilet KALMADI!");
13            return false;
14        }
15    }
16 }
17
```

```
18 // Görev sınıfı: Gişe (sürekli bilet almaya çalışır)
19 class Gişe extends Thread {
20     private String ad;
21     private BiletHavuzu havuz;
22
23     public Gişe(String ad, BiletHavuzu havuz) {
24         this.ad = ad;
25         this.havuz = havuz;
26     }
27
28     @Override
29     public void run() {
30         // Her gişe 5 kez bilet almaya çalışsın (yeterli)
31         for (int i = 1; i <= 5; i++) {
32             boolean basarili = havuz.biletAl(ad);
33             if (!basarili) break; // bilet kalmadıysa çık
34
35             try { Thread.sleep(50); } catch (InterruptedException e) {} // satış işlemi simülasyonu
36         }
37         System.out.println(ad + " işlemini tamamladı.");
38     }
39 }
```

```
41 // Ana sınıf
42 public class BiletSatisSimulasyonu {
43     public static void main(String[] args) {
44         BiletHavuzu havuz = new BiletHavuzu();
45
46         Gise gise1 = new Gise("Gişe-1 (Soldaki)", havuz);
47         Gise gise2 = new Gise("Gişe-2 (Ortadaki)", havuz);
48         Gise gise3 = new Gise("Gişe-3 (Sağdaki)", havuz);
49
50         gise1.start();
51         gise2.start();
52         gise3.start();
53     }
54 }
55
```